
BACKEND & SYSTEM IMPROVEMENTS

1. Selective Disclosure for Credentials — Right now if you share a credential, the other person sees everything in it. Like if someone just needs to check your age, they'd still end up seeing your full passport number, which is not great. Omnimise should let users prove specific claims — like "I'm over 21" or "I graduated from IIT Bombay" — without revealing the full document. This can be done by breaking credentials into individually signed fields, so you only share the exact fields you want to.

2. Telemetry Stack Cleanup CHARLES PROXY FINDING — So I intercepted the app's network traffic using Charles Proxy and found that it's talking to a bunch of third-party services at the same time — ad trackers, multiple analytics tools, and error logging services. For an app that's supposed to be all about data privacy, sending user data through ad trackers doesn't really make sense. It's also leaking the user's IP address by calling an external API. The fix isn't that complicated: switch to one self-hosted analytics endpoint, remove the ad tracker calls entirely, and just get the IP from the server request headers instead of a third-party call.

3. Local-First Device Sync — Since there's no central database, syncing your credentials across phone, smart card, and desktop could easily run into conflicts or even data loss. A good approach would be to use peer-to-peer syncing (like how WebRTC lets devices communicate directly) so your devices can sync with each other over local WiFi or Bluetooth, encrypted, without needing any cloud server in between.

FRONTEND & UI/UX IMPROVEMENTS

1. Interactive Consent Sheet for Selective Sharing — When you share a credential right now, it feels all-or-nothing, which can make users uncomfortable. A nicer approach would be a toggle-based permission sheet (kind of like how iOS asks for permissions) where you can check or uncheck individual fields — like showing "Company Name" and "Role" but hiding "Salary." You could even add a trust score indicator that shows how the verification strength changes as you toggle fields on and off.

2. Haptic Feedback and Verification Animations for Physical Taps — When you tap the Omnimise Card or Ring at a checkpoint, there's no real feedback — no vibration, no animation, nothing. It feels kinda dead. Adding a nice glassmorphic success popup along with different vibration patterns (like a quick double-buzz for success, a long vibration for retry) would make the whole interaction feel much more responsive and polished.

3. Live Resume Editor with Drag-and-Drop — The resume generation is automated which is cool, but tweaking it means you have to keep exporting and re-exporting. It would be way better to have a split-screen editor — your credential blocks on the left side, and a live PDF preview on the right. You could drag and drop sections, reorder stuff, toggle things on and off, and the preview updates in real time. A QR code in the footer would link back to the verified version.

FULL-STACK FEATURE SUGGESTIONS

1. Verified BGV Portal for HR Teams — Basically a dashboard for recruiters where they can set what they want to verify (e.g., "3 years at Stripe as an Engineer"). The candidate gets a push notification on their phone, approves the specific fields from their app, and the recruiter's dashboard shows a green "Verified" checkmark. The important part: no candidate personal data is stored on the company's servers.

2. Family Loops: Shared Encrypted Vaults — Think of it as a shared folder but with proper encryption. You'd see family member profiles and shared document folders in a drag-and-drop interface. The encryption keys are managed so that only members added to the group can decrypt the files. Files are stored in the cloud, but only the family members in the loop have access.

3. Rotating QR Codes for In-Person Verification — For in-person verification (like when someone is doing a background check face-to-face), the app shows a rotating QR code that refreshes every 30 seconds with a countdown timer around it. A WebSocket connection handles the real-time handshake between the scanner and the app, but the actual credential data is end-to-end encrypted so even the backend can't see what's being shared.